

Stack

LIFO Data Structure

insert order $\rightarrow$ [2 3 5 10]



[10 5 3 2]

Operation

- 1) push() $\leftarrow$ last in
- 2) pop() $\rightarrow$ top out
- 3) peek() $\rightarrow$ top print

implementation

① Array

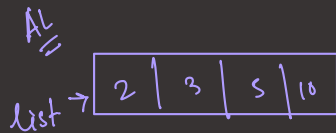
$\downarrow$
fixed size $\rightarrow$ problem

✓ ② ArrayList

$\downarrow$
dynamic
[Ok-ok]

✓ ③ Linked List.

$\downarrow$
Dynamic
 $\downarrow$
Convenient for developer

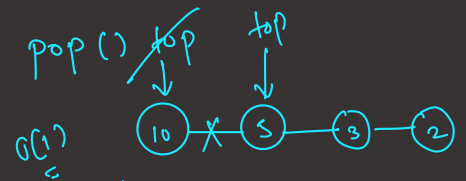
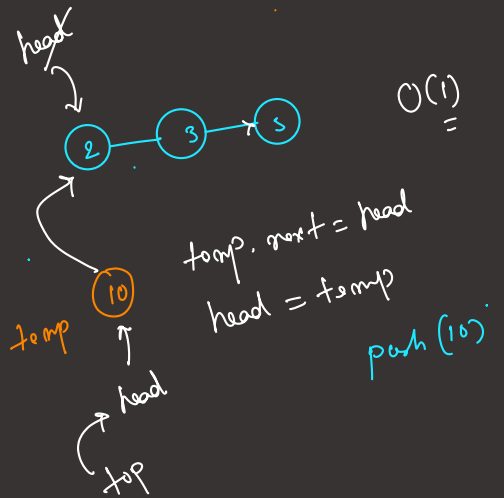


push(15) $O(1)$

$\uparrow$
list.add();

pop: list.remove() $O(1)$

peek: list.get(size()-1) $O(1)$



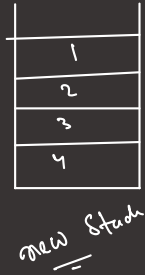
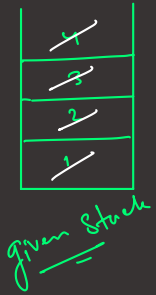
JCF

Stack < Integer > s = new Stack < > ();

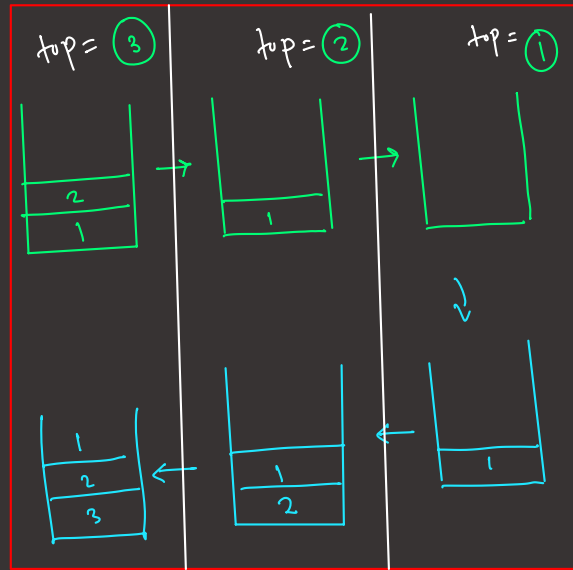
Question 3

reverse a stack

Way 1



Way 2



top = s.pop();

reverse() // recursive call

pushAtBottom(top);

Question 4

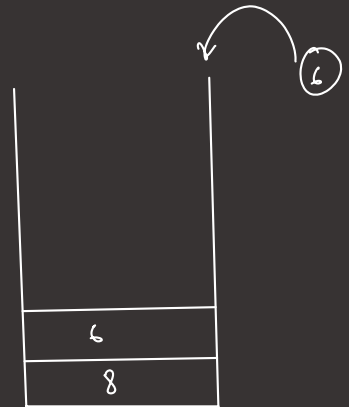
Next greater Element [from right]

Next greater element of some element x in an array is the first greater element that is to the right of the x in the same array.

arr = [6, 8, 0, 1, 3, 2, 5]

nextGreater = [8, -1, 1, 3, 5, 5, -1]

→ [8, -1, 1, 3, 5, 5, -1]



nextGreater [arr.length]

Stack<Integer> s

for (int i = arr.length - 1 to 0) {

while (!s.isEmpty() && arr[s.peek()] <= arr[i]) {

s.pop();

}

if (s.isEmpty()) nextGreater[i] = -1; }



else {

arr[greater[i]] = arr[s.peek()] ;

}

s.push(i) ;

}

Question 5

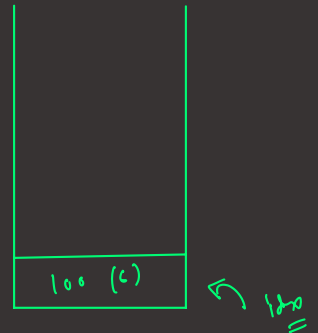
Stack Span problem

Stock price = $[100, 80, 60, 70, 60, 85, 100]$
 $\begin{matrix} & & & & & \checkmark & \\ & & & & & 5 & \\ \rightarrow & 0 & 1 & 2 & 3 & 4 & 5 & 6 \end{matrix}$

find out: max no. of consecutive days for which
price $\leq$ today's price.

Span o/p:

✓	✓	✓	✓	✓	✓	✓	
1	1	1	2	1	5	7	



100

prevHigh[] =

-1	0	1	1	3	0	-1	
0	1	2	3	4	5	6	

PH =

-1	0	1	1	3	0	-1	
----	---	---	---	---	---	----	--

$$\text{Span} \quad \boxed{\text{day } i = i - \text{prevHigh}[i]}$$

$$\text{day } 0 = 0 - (-1) = 1$$

$$\text{day } 1 = 1 - 0 = 1$$

$$\text{day } 2 = 2 - 1 = 1$$

$$\text{day } 3 = 3 - 1 = 2$$

$$\text{day } 4 = 4 - 3 = 1$$

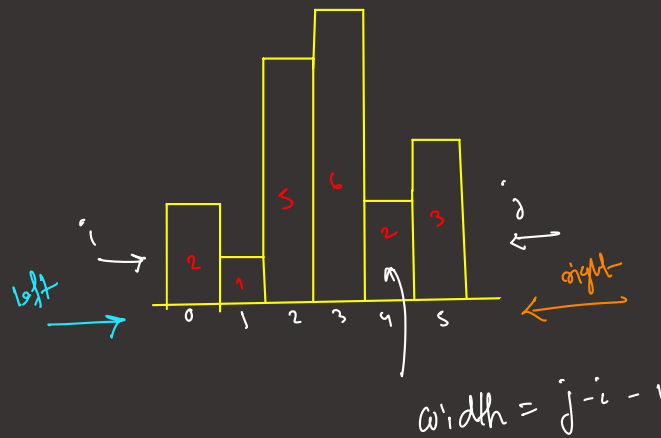
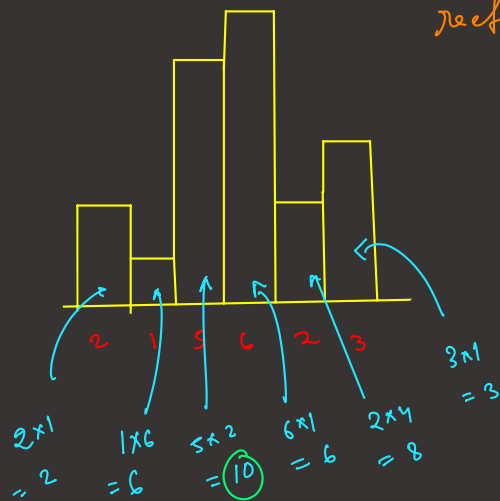
$$\text{day } 5 = 5 - 0 = 5$$

$$\text{day } 6 = 6 - (-1) = 7$$

Question 6

Max Area in a Histogram. height = [2, 1, 5, 6, 2, 3]

Given an array of integer heights representing the histogram's bar height where the width of each bar is 1. Return the area of the largest rectangle in the histogram.



(i) next smaller left = [-1 | -1 | 1 | 2 | 1 | 4]
 (j) next smaller right = [1 | 6 | 4 | 4 | 6 | 6]

$n = \text{arr.length}$

$$\text{width}[0] = 1 - (-1) - 1 = 2 - 1 = 1$$

$$\text{width}[1] = 6 - (-1) - 1 = 6 + 1 - 1 = 6$$

Question 7 Valid Parentheses

input string $\{ ' (,) ' , ' [,] ' , ' { , } ' \}$ $\xrightarrow{\text{Valid ?}}$

- valid (i) open brace () closed brace of same type.
 (ii) order maintain ({ [] })
 (iii)) (X not valid

$S_1 = "() [] \{ \}"$ ✓
 $S_2 = "()"$ ✓
 $S_3 = "{ [()] () }"$ ✓
 $S_4 = "()"$ X

Approach



str = "{ [()] () } { () }"
 ↓ ↓ ↓ ↓ ↓
 { [()] () } { () }

```

public static boolean isValidParenthesis(String str){
    Stack<Character> s = new Stack<>();

    for(int i=0; i<str.length(); i++){
        char ch = str.charAt(i);

        // open Bracket
        if(ch == '(' || ch == '{' || ch == '['){
            s.push(ch);
        }
        // closing Bracket
        else if(!s.isEmpty() && (ch == ')' && s.peek() == '(') ||
                (ch == '}' && s.peek() == '{') ||
                (ch == ']' && s.peek() == '[')){
            s.pop();
        } else{
            return false;
        }
    }

    if(!s.isEmpty()){
        return false;
    } else{
        return true;
    }
}

```

for (i = 0 to str.length)

(i) i == str.length but s.isEmpty not true
false

(ii) s.isEmpty() and char(i) → closing bracket
false

(iii) s.peek() ≠ corresponding closing brackets
false

o/s → true

Question 8 Duplicate Parenthesis

A set of parentheses are duplicate if the same expression is surrounded by multiple parentheses.

✓
 $((((a+b)) + (c+d)))$ ✗
 $((a+(b)) + (c+d))$ ✓
 $((a+b) + (c+d))$ ✓

(i) opening bracket ' ('
 (ii) operator +, -, *, /
 (iii) operands (a, b, c, ...)
 (iv) closing bracket ')' ')'
 s.push()

count < 1

no. of element

count = 0

s.pop until opening bracket comes. } → count++

(count < 1) { false }

else { s.pop() }

a
b
c
d

```
public static boolean isDuplicate(String str){
    Stack<Character> s =new Stack<>();

    for(int i=0; i<str.length(); i++){
        char ch = str.charAt(i);

        if(ch != ')' ){
            s.push(ch);
        }else{ // closing bracket
            int count = 0;
            while (s.peek()!='(') {
                s.pop();
                count++;
            }

            if(count<1){
                return true;
            }else{
                s.pop();
            }
        }
    }

    return false;
}
```